Neuralwave Hackathon – Technical Report

Neurowave 2026

Reconstruction of 15-minute electricity price profiles across multiple European markets

Team Tom&Will

William Hvid Larsen Amura Raja Wishwanath Goud Arne Christian Hestermann
Sebastian Hrnciarik Hai Phong Nguyen

October 18, 2026

Team	Team Tom&Will
Track	Reconstruction of 15-minute electricity price profiles across multiple European markets
Repository	`https://github.com/your-team/your-repo`
Demo	`https://your-demo-link.example`
Video	`https://your-video-link.example`

0.1 Problem Statement

Transform an already available daily hourly curve into a realistic quarter-hourly profile, while maintaining exact consistency with the 24 reference hourly prices.

0.2 Objectives

Design an implement a model which accurately predicts quarter-hourly prices with good explainability.

1 Solution Overview

We deliver an XGBoost (Extreme Gradient Boosting) model to predict the quarter-hourly price predictions. We evaluated variety of machine learning methods, and although our final solution is amongst the simplest, we observed strong performance and generalisibility.

XGBoost is a tree-based supervision machine learning algorithm. good generalisibility incorporates high regularization, which is helpful with our limited dataset situation.

1.1 Key Features

- Explainability
- Lightweight training
- Generalisibility

2 Methodology

2.1 Data

We extracted 86 features in total, amongst them delivery_date, market_id, zone, hour_of_day, hour_price, and lag_1d_q1_dev for example. We focused on providing information about repeating patterns, such as daily or weekly trends by calculating and using descriptive statistics, for example standard deviation or mean. We observed no drop in performance by including high amount of features, as XGBoost decision trees are free to not use some of them.

2.2 XGFactor

We fit an XGFactor on our training data after extracting the features. Selected hyperparameter values are shown in the table below.

We implement one additional modification. After the model predicts how far each quarter-hour lies from the hourly price, all four deviations are multiplied by one fixed factor slightly below one. The hourly average is unchanged, because the deviations still sum to zero.

Parameter	Value
Number of trees (`n_estimators`)	800
Learning rate	0.02
Maximum tree depth (`max_depth`)	9
Minimum child weight (`min_child_weight`)	3
Row subsampling (`subsample`)	0.8
Column subsampling (`colsample_bytree`)	0.7
Objective	`reg:absoluteerror` (L1, matches the MAE metric)
Tree method	`hist`
Random seed	42
Regressors	4 (one per quarter-hour)
Ramp prior strength k	2.0
Hour-boundary smoothing	0.02
Shrink factor	0.875 (chosen on the last 28 training days, then fixed)

Table 1: XGBoost configuration of the submitted model.

The factor is there because the intra-hour price pattern weakened over the year. A model trained on older months therefore tends to exaggerate the pattern on new days, and shrinking the deviations corrects this. It also makes the predictions a little more cautious, which lowers the error when the model is uncertain.

The factor is chosen once during training, on the most recent training weeks, and then kept fixed

2.3 LSTM sequence prediction with 1D Convolution Encoder

The `ConvLSTMUpsampler` is a hybrid deep learning model designed for high-resolution time series forecasting (e.g., upsampling hourly data to 15-minute intervals). It combines a Temporal Convolutional Network (TCN) encoder with an LSTM-based decoder to generate structured multi-step predictions.

1. Encoder: TCN processing historical quarter-data via causal convolutions, GroupNorm, ReLU, adaptive pooling, and a tanh projection to initialize the LSTM hidden state (h_0).

2. Decoder: LSTM processing hourly features, conditioned on the encoder's initial hidden state (h_0) and zero cell state (c_0).

3. Head: Linear projection producing raw quarter-hour adjustments (offsets).

4. Output: Zero-centered mean offsets added to the base hourly price to produce the final flattened 96-step quarter-hour predictions.

Figure 1: Overview of the ConvLSTMUpsampler architecture pipeline.

2.4 Training and Evaluation Setup

We used data from the month of August as held out validation set for evaluation of all our methods. Below is the summary of achieved performances. Base this table, we propose XGBCrossZone with fixed factor as the final method.

Table 2: Performance comparison of different models evaluated on held-out data from August using Mean Absolute Error (MAE).

Method	**Dev MAE**
Flat baseline	8.773
Interpolation	6.238
LSTM+1DConv	4.561
CSB-CatBoost	4.322
XGBCrossZone	4.292
XGBCrossZone + fixed factor (submitted)	4.272

3 Results

Present quantitative and qualitative results.

Method	**Metric A**
Baseline	8.773
Ours	**4.272**

Table 3: Comparison against the baseline.

4 Challenges and Limitations

4.1 Data

- **One year of data.** The training set covers 363 days (Oct 2025 – Sep 2026). With a single year we cannot separate a lasting trend from seasonal effects, and the neural models had only about 1,150 day–zone samples to learn from.
- **A new and changing market.** The day-ahead market moved to 15-minute products on 1 October 2025, exactly when the data starts. The intra-hour pattern weakened over the year (the ramp inside an hour per EUR/MWh of hourly slope fell from about 0.32 to 0.24), while volatility rose; September 2026 was the most volatile month in the data. Models trained on older months therefore overestimate the pattern on new days.
- **Anonymised zones.** The four zones are anonymous, so no geographic or market-coupling knowledge could be used, only what the prices themselves show.
- **Extreme prices.** Prices range from roughly -500 to 750 EUR/MWh, with negative prices and sudden spikes, which makes scaling and learning harder.
- **No external drivers.** Load, generation and weather data are not allowed, so the model cannot see the causes of sudden price moves.

4.2 What the model cannot predict

- **Single-quarter spikes.** The error is concentrated in a few quarters: the worst 10% of quarters account for about 41% of the total error. Many of these spikes are not visible in the hourly curve and are essentially unpredictable from the permitted inputs.

- **Ramps and hard zones.** Errors are largest in the morning (09–10) and evening (17–21) ramps and in ZONE_1 and ZONE_2.

4.3 Validation

- **Random splits were misleading.** Splits that train on days both before and after a validation day hide the drift and overstate accuracy. We moved to a forward-in-time rolling split, which matches the official setting but leaves fewer days to validate on.
- **Small differences are noise.** With 30–46 validation days per period, differences below about 0.02 EUR/MWh cannot be distinguished from noise, which limits fine-tuning.

4.4 Constraints from the rules

- A version that re-estimated the shrink factor every day from recent history scored slightly better, but it would recalibrate a parameter after the freeze, which the rules forbid. We use a factor fixed at training time instead.
- The fixed factor assumes the pattern keeps weakening. If it became stronger again, the factor would shrink predictions slightly too much.

4.5 Engineering issues

- **Look-ahead in a feature pipeline.** An automated test revealed that the k-NN features of one model could see earlier hours of the same day during training. We fixed it and added the test to the repository.
- **Robustness.** Models relying on history degrade when it is missing (LightGBM went from 4.10 to 5.43 MAE with no history). A missing zone originally crashed the pipeline; it now runs, but the cross-zone features are then missing and accuracy drops.
- **Library conflicts.** PyTorch and XGBoost crashed when loaded in the same process on macOS (conflicting OpenMP libraries), which made combining neural and tree models harder.
- **Time.** A blend of XGBoost, CatBoost and a CNN was about 1% better, but it was not submitted: the extra complexity and risk were not justified by the gain within the deadline.

.